

Data Representation and Numerical Accuracy

Encoding Text, Numbers, and the Limits of Computation

Martin Benning

University College London

ENGF0034 – Design and Professional Skills

Recap and Introduction

Recap from Previous Lecture (Week 6)

We explored persistent storage (HDDs/SSDs) and the Operating System's abstraction: the File System. We established that a file is fundamentally a sequence of bytes, and learned how to perform File I/O in Python.

This Week's Focus: Interpretation

If a file is just bytes (numbers 0-255), how do we represent complex information like text in different languages, large integers, negative numbers, and real numbers?

Objectives

To understand the standards for data encoding (Unicode, Two's Complement, IEEE 754) and the critical implications of these representations for numerical accuracy and error analysis.

Recap and Introduction

Recap from Previous Lecture (Week 6)

We explored persistent storage (HDDs/SSDs) and the Operating System's abstraction: the File System. We established that a file is fundamentally a sequence of bytes, and learned how to perform File I/O in Python.

This Week's Focus: Interpretation

If a file is just bytes (numbers 0-255), how do we represent complex information like text in different languages, large integers, negative numbers, and real numbers?

Objectives

To understand the standards for data encoding (Unicode, Two's Complement, IEEE 754) and the critical implications of these representations for numerical accuracy and error analysis.

Recap and Introduction

Recap from Previous Lecture (Week 6)

We explored persistent storage (HDDs/SSDs) and the Operating System's abstraction: the File System. We established that a file is fundamentally a sequence of bytes, and learned how to perform File I/O in Python.

This Week's Focus: Interpretation

If a file is just bytes (numbers 0-255), how do we represent complex information like text in different languages, large integers, negative numbers, and real numbers?

Objectives

To understand the standards for data encoding (Unicode, Two's Complement, IEEE 754) and the critical implications of these representations for numerical accuracy and error analysis.

Lecture Overview

- 1 File Formats and Interpretation
- 2 Character Encoding
- 3 Representing Integers
- 4 Representing Real Numbers: Floating Point
- 5 Numerical Errors and Machine Arithmetic
- 6 Data Serialisation

File Formats and Interpretation

File Formats and Interpretation

Interpreting Bytes: Binary vs. Text

It is the responsibility of the application to interpret file bytes according to a specific format.
We generally categorise files into two types:

Binary Files

- Designed for machine consumption.
- Often contain raw memory dumps, compressed data, or proprietary formats.
- Not typically human-readable in a standard text editor.
- Examples: '.exe', '.jpg', legacy '.doc'.

Interpreting Bytes: Binary vs. Text

Binary Files

- Designed for machine consumption.
- Often contain raw memory dumps, compressed data, or proprietary formats.
- Not typically human-readable in a standard text editor.
- Examples: '.exe', '.jpg', legacy '.doc'.

Text Files

- Designed to be human-readable.
- Bytes are interpreted using a specific **character encoding** standard.
- Examples: '.py', '.xml', '.txt'.

Case Study: Microsoft Word (.doc vs .docx)

Microsoft Word illustrates the evolution of file formats.

Legacy .doc (Binary)

- A complex, proprietary binary format.
- The video demonstrated that saving "Hello World" yielded a 22KB file, containing extensive metadata and internal structures.
- **Risk:** Can hide sensitive information (e.g., deleted comments in Track Changes).

Case Study: Microsoft Word (.doc vs .docx)

Legacy .doc (Binary)

- A complex, proprietary binary format.
- The video demonstrated that saving "Hello World" yielded a 22KB file, containing extensive metadata and internal structures.
- **Risk:** Can hide sensitive information (e.g., deleted comments in Track Changes).

Modern .docx (Structured Text)

- A documented, open standard.
- It is actually a **ZIP archive**.
- Content is stored primarily in **XML** files (which are text-based).
- More robust and interoperable.

The Nuances of "Plain Text": Line Endings

Even "plain text" is not universal due to historical conventions for marking the end of a line.

Operating System Conventions

Windows/DOS: Uses Carriage Return + Line Feed (`\r\n` or CR LF).

Unix/Linux: Uses Line Feed only (`\n` or LF).

Modern macOS (OS X+): Also uses LF (`\n`), as it is Unix-based.

(Historical) Classic Mac OS: Used CR only (`\r`).

Implication

To correctly interpret any file, you must know its format and encoding conventions, including the expected line endings. Modern editors often handle this automatically, but it remains a potential source of errors.

The Nuances of "Plain Text": Line Endings

Even "plain text" is not universal due to historical conventions for marking the end of a line.

Operating System Conventions

Windows/DOS: Uses Carriage Return + Line Feed (`\r\n` or CR LF).

Unix/Linux: Uses Line Feed only (`\n` or LF).

Modern macOS (OS X+): Also uses LF (`\n`), as it is Unix-based.

(Historical) Classic Mac OS: Used CR only (`\r`).

Implication

To correctly interpret any file, you must know its format and encoding conventions, including the expected line endings. Modern editors often handle this automatically, but it remains a potential source of errors.

Character Encoding

Character Encoding

Prerequisite: The Hexadecimal Shorthand

Why do we use "0x..."?

Binary is verbose (10100011), and Decimal doesn't map cleanly to bits. **Hexadecimal (Base-16)** is the standard shorthand because **1 Hex digit represents exactly 4 bits**.

Decimal	Binary	Hex	Decimal	Binary	Hex
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
...	...	...	15	1111	F

Key Takeaway

When you see 0x0403, read it as two bytes: 04 and 03.

The Evolution of Character Encoding

The Goal

To represent human-readable characters as numerical values (bytes).

Stage 1: ASCII (1960s)

American Standard Code for Information Interchange.

- A **7-bit** code ($2^7 = 128$ characters).
- Covers English alphabet, digits, and punctuation.
- **Limitation:** Highly Anglo-centric. Lacks support for accents (é), many currencies (£, €), and non-Latin scripts.

The Evolution of Character Encoding

The Goal

To represent human-readable characters as numerical values (bytes).

Stage 1: ASCII (1960s)

American Standard Code for Information Interchange.

- A **7-bit** code ($2^7 = 128$ characters).
- Covers English alphabet, digits, and punctuation.
- **Limitation:** Highly Anglo-centric. Lacks support for accents (é), many currencies (£, €), and non-Latin scripts.

The Evolution of Character Encoding

Stage 2: ISO 8859 (Extended ASCII)

Utilized the unused 8th bit, extending the range to $2^8 = 256$ characters.

- Kept the lower 128 values for ASCII compatibility.
- Used the upper 128 values (128-255) for additional characters.

The Problem: Fragmentation (Code Pages)

128 extra characters are insufficient for all languages. This led to different variants (Code Pages):

- **ISO 8859-1 (Latin-1)**: Western European.
- **ISO 8859-7**: Greek.

A file encoded in one variant would look corrupted (Mojibake) if opened assuming another.

The Evolution of Character Encoding

Stage 2: ISO 8859 (Extended ASCII)

Utilized the unused 8th bit, extending the range to $2^8 = 256$ characters.

- Kept the lower 128 values for ASCII compatibility.
- Used the upper 128 values (128-255) for additional characters.

The Problem: Fragmentation (Code Pages)

128 extra characters are insufficient for all languages. This led to different variants (Code Pages):

- **ISO 8859-1 (Latin-1):** Western European.
- **ISO 8859-7:** Greek.

A file encoded in one variant would look corrupted (Mojibake) if opened assuming another.

The Universal Solution: Unicode

Stage 3: Unicode

A universal standard aiming to assign a unique number to every character in every script.

Code Points

These unique numbers are called **Code Points**, typically written in hexadecimal (U+...).

- 'A': U+0041
- '£': U+00A3
- 'π': U+03C0
- Smiley Emoji: U+1F600

The Universal Solution: Unicode

Stage 3: Unicode

A universal standard aiming to assign a unique number to every character in every script.

Code Points

These unique numbers are called **Code Points**, typically written in hexadecimal (U+...).

- 'A': U+0041
- '£': U+00A3
- 'π': U+03C0
- Smiley Emoji: U+1F600

The Universal Solution: Unicode

The Scope

Unicode defines over 140,000 characters, requiring more than 16 bits (it uses up to 21 bits).

The Distinction

Unicode defines the *mapping* (the Code Point). We still need an *encoding* method to store these Code Points efficiently in memory/files.

The Universal Solution: Unicode

The Scope

Unicode defines over 140,000 characters, requiring more than 16 bits (it uses up to 21 bits).

The Distinction

Unicode defines the *mapping* (the Code Point). We still need an *encoding* method to store these Code Points efficiently in memory/files.

The Standard Encoding: UTF-8

The Challenge

Storing every character using a fixed 32 bits (UTF-32) is simple but wasteful. We need efficiency and ASCII compatibility.

UTF-8 (Unicode Transformation Format - 8-bit)

The dominant standard today. It is a variable-length encoding (1 to 4 bytes per character).

The Standard Encoding: UTF-8

The Challenge

Storing every character using a fixed 32 bits (UTF-32) is simple but wasteful. We need efficiency and ASCII compatibility.

UTF-8 (Unicode Transformation Format - 8-bit)

The dominant standard today. It is a variable-length encoding (1 to 4 bytes per character).

The Standard Encoding: UTF-8

UTF-8 (Unicode Transformation Format - 8-bit)

The dominant standard today. It is a variable-length encoding (1 to 4 bytes per character).

Design Features

ASCII Compatibility: Standard ASCII characters (U+0000 to U+007F) are stored in a single byte, exactly as in ASCII.

Efficiency: Common characters use fewer bytes.

Self-Synchronization: The structure allows easy detection of character boundaries.

How UTF-8 Works: Variable Length

Bytes	Starts With (Binary)	Structure (x = payload bits)
1	0... (ASCII)	0xxxxxxx
2	110...	110xxxxx 10xxxxxx
3	1110...	1110xxxx 10xxxxxx 10xxxxxx
4	11110...	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

Key Mechanism

- The number of leading '1's in the first byte indicates the total length of the sequence.
- All subsequent bytes (Continuation Bytes) start with 10.

How UTF-8 Works: Variable Length

Bytes	Starts With (Binary)	Structure (x = payload bits)
1	0... (ASCII)	0xxxxxxx
2	110...	110xxxxx 10xxxxxx
3	1110...	1110xxxx 10xxxxxx 10xxxxxx
4	11110...	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

Key Mechanism

- The number of leading '1's in the first byte indicates the total length of the sequence.
- All subsequent bytes (Continuation Bytes) start with 10.

How UTF-8 Works: Variable Length

Key Mechanism

- The number of leading '1's in the first byte indicates the total length of the sequence.
- All subsequent bytes (Continuation Bytes) start with 10.

Benefit: Self-Synchronisation

By looking at any byte, we can instantly know if it is the start of a character or a continuation byte, making the encoding robust against corruption or random access.

Unicode in Python I

Python 3 strings are natively Unicode. Encoding/decoding happens during I/O.

Reading and Writing Files

When opening a file in text mode ('rt' or 'wt'), Python handles the encoding/decoding. It is best practice to explicitly specify UTF-8.

Listing 1: unicode_example.py (Conceptual Illustration)

```
1 text = "Hello π" # (Pi symbol U+03C0)
2
3 # Write the string to a file using UTF-8 encoding
4 # with open('unicode_data.txt', 'wt', encoding="utf-8") as outfile:
5 #     outfile.write(text)
```

Unicode in Python II

```

6
7 # Read the file back in text mode (decodes UTF-8 to string)
8 # with open('unicode_data.txt', 'rt', encoding="utf-8") as infile:
9 #     content = infile.read()
10 # content is: "Hello π"
11
12 # Read the file in binary mode (returns raw bytes)
13 # with open('unicode_data.txt', 'rb') as infile:
14 #     raw_bytes = infile.read()
15 # raw_bytes is: b'Hello \xcf\x80' (Pi is 2 bytes in UTF-8)

```

Representing Integers

Representing Integers

From Abstraction to Hardware

Python's Abstraction: Arbitrary Precision

In Python, integers can be arbitrarily large. Python automatically manages the memory required (internally storing them as arrays of digits).

The Hardware Reality: Fixed Size

CPUs operate on fixed-size integers (e.g., 32-bit or 64-bit).

The Requirement for Serialisation

When storing integers to a file or transmitting them over a network, we must convert Python's representation into a specific sequence of raw bytes. We must define the format.

From Abstraction to Hardware

Python's Abstraction: Arbitrary Precision

In Python, integers can be arbitrarily large. Python automatically manages the memory required (internally storing them as arrays of digits).

The Hardware Reality: Fixed Size

CPUs operate on fixed-size integers (e.g., 32-bit or 64-bit).

The Requirement for Serialisation

When storing integers to a file or transmitting them over a network, we must convert Python's representation into a specific sequence of raw bytes. We must define the format.

From Abstraction to Hardware

Python's Abstraction: Arbitrary Precision

In Python, integers can be arbitrarily large. Python automatically manages the memory required (internally storing them as arrays of digits).

The Hardware Reality: Fixed Size

CPUs operate on fixed-size integers (e.g., 32-bit or 64-bit).

The Requirement for Serialisation

When storing integers to a file or transmitting them over a network, we must convert Python's representation into a specific sequence of raw bytes. We must define the format.

Defining an Integer Representation

To serialise an integer, we must define three parameters:

1. Size (Number of Bytes)

How many bytes will we use? (e.g., 1, 2, 4, 8). This determines the range of values. Choosing too small a size leads to **Overflow**.

2. Byte Order (Endianness)

For multi-byte integers, in what order are the bytes stored in memory?

3. Signedness

How do we represent negative numbers?

Defining an Integer Representation

To serialise an integer, we must define three parameters:

1. Size (Number of Bytes)

How many bytes will we use? (e.g., 1, 2, 4, 8). This determines the range of values. Choosing too small a size leads to **Overflow**.

2. Byte Order (Endianness)

For multi-byte integers, in what order are the bytes stored in memory?

3. Signedness

How do we represent negative numbers?

Defining an Integer Representation

To serialise an integer, we must define three parameters:

1. Size (Number of Bytes)

How many bytes will we use? (e.g., 1, 2, 4, 8). This determines the range of values. Choosing too small a size leads to **Overflow**.

2. Byte Order (Endianness)

For multi-byte integers, in what order are the bytes stored in memory?

3. Signedness

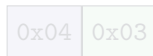
How do we represent negative numbers?

Endianness: The Order of Bytes

Consider the 16-bit integer 1027 (Decimal). Hexadecimal: 0x0403. It consists of two bytes: 0x04 (Most Significant Byte, MSB) and 0x03 (Least Significant Byte, LSB).

Big Endian

The MSB is stored first (at the lowest memory address).



Addr A A+1

Little Endian

The LSB is stored first.



Endianness: The Order of Bytes

Big Endian

The MSB is stored first (at the lowest memory address).



Addr A A+1

Little Endian

The LSB is stored first.



Addr A A+1

Endianness: The Order of Bytes

Big Endian

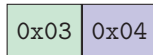
The MSB is stored first (at the lowest memory address).



Addr A A+1

Little Endian

The LSB is stored first.



Addr A A+1

Endianness in Practice

The Architectural Divide

- **Little Endian:** Dominant today (Intel/AMD x86/x64, most ARM configurations).
- **Big Endian:** Used by some older architectures.

Network Byte Order

To ensure compatibility between different systems communicating over the internet, a standard was defined: **Network Byte Order is Big Endian.**

The Consequence

Modern (little endian) computers must actively swap the byte order when sending or receiving multi-byte integers over a network.

Endianness in Practice

The Architectural Divide

- **Little Endian:** Dominant today (Intel/AMD x86/x64, most ARM configurations).
- **Big Endian:** Used by some older architectures.

Network Byte Order

To ensure compatibility between different systems communicating over the internet, a standard was defined: **Network Byte Order is Big Endian.**

The Consequence

Modern (little endian) computers must actively swap the byte order when sending or receiving multi-byte integers over a network.

Endianness in Practice

The Architectural Divide

- **Little Endian:** Dominant today (Intel/AMD x86/x64, most ARM configurations).
- **Big Endian:** Used by some older architectures.

Network Byte Order

To ensure compatibility between different systems communicating over the internet, a standard was defined: **Network Byte Order is Big Endian.**

The Consequence

Modern (little endian) computers must actively swap the byte order when sending or receiving multi-byte integers over a network.

Representing Negative Numbers

How do we store a negative integer? (e.g., -13).

Historical Methods (and their flaws)

Sign and Magnitude: MSB is the sign bit (1=negative).

- **Problem:** Two zeros (+0 and -0); complex arithmetic hardware.

One's Complement: Invert all bits of the positive number.

- **Problem:** Still two zeros.

The Modern Standard: Two's Complement

To negate a number: Invert all bits **and add one**.

Representing Negative Numbers

How do we store a negative integer? (e.g., -13).

Historical Methods (and their flaws)

Sign and Magnitude: MSB is the sign bit (1=negative).

- **Problem:** Two zeros (+0 and -0); complex arithmetic hardware.

One's Complement: Invert all bits of the positive number.

- **Problem:** Still two zeros.

The Modern Standard: Two's Complement

To negate a number: Invert all bits **and add one**.

The Advantages of Two's Complement

Example: -13 (8-bit)

- 1 Start with +13: 00001101
- 2 Invert bits: 11110010
- 3 Add 1: 11110011 (-13)

Verification: 13 + (-13)

```

    00001101  (+13)
+   11110011  (-13)
-----

```

```

(1)00000000  (0)

```

(The overflow bit (1) is discarded in fixed-width arithmetic).

The Advantages of Two's Complement

Example: -13 (8-bit)

- 1 Start with +13: 00001101
- 2 Invert bits: 11110010
- 3 Add 1: 11110011 (-13)

Key Advantages

- **Unique Zero:** Only one representation for zero.
- **Simplified Hardware:** Standard addition and subtraction circuits work identically for both unsigned and signed numbers.

Verification: $13 + (-13)$

The Advantages of Two's Complement

Example: -13 (8-bit)

- 1 Start with +13: 00001101
- 2 Invert bits: 11110010
- 3 Add 1: 11110011 (-13)

Verification: 13 + (-13)

```

    00001101  (+13)
+   11110011  (-13)
-----
(1)00000000  (0)

```

(The overflow bit (1) is discarded in fixed-width arithmetic).

Integers in Python: Serialisation (ints.py)

When writing integers to binary files, we use `.to_bytes()` to explicitly define representation.

```
1 i = 1027 # Hex: 0x0403
2 # 1027 as 2 bytes, big endian, unsigned
3 b_big = i.to_bytes(2, byteorder='big')
4 # Result: b'\x04\x03'
5 # 1027 as 4 bytes, little endian, unsigned
6 b_little = i.to_bytes(4, byteorder='little')
7 # Result: b'\x03\x04\x00\x00'
8 # -1027 as 4 bytes, big endian, signed (Two's Complement)
9 b_neg = (-1027).to_bytes(4, byteorder='big', signed=True)
10 # Result: b'\xff\xff\xfb\xfd'
```

Error Handling

Python will raise an `OverflowError` if the number cannot fit into the specified size and signedness.

Integers in Python: Serialisation (ints.py)

When writing integers to binary files, we use `.to_bytes()` to explicitly define representation.

```
1 i = 1027 # Hex: 0x0403
2 # 1027 as 2 bytes, big endian, unsigned
3 b_big = i.to_bytes(2, byteorder='big')
4 # Result: b'\x04\x03'
5 # 1027 as 4 bytes, little endian, unsigned
6 b_little = i.to_bytes(4, byteorder='little')
7 # Result: b'\x03\x04\x00\x00'
8 # -1027 as 4 bytes, big endian, signed (Two's Complement)
9 b_neg = (-1027).to_bytes(4, byteorder='big', signed=True)
10 # Result: b'\xff\xff\xfb\xfd'
```

Error Handling

Python will raise an `OverflowError` if the number cannot fit into the specified size and signedness.

Representing Real Numbers: Floating Point

Representing Real Numbers: Floating Point

Beyond Integers: Fractional Numbers

How do we store numbers that are not integers (e.g., 0.25, π)?

The Binary Point

We use a binary point like a decimal point.

- Columns right of the point: 2^{-1} (0.5), 2^{-2} (0.25), ...

Example: 2.25 (Decimal) = 10.01 (Binary).

Fixed-Point vs. Floating-Point

- **Fixed-Point:** Fix the location of the binary point. Limited range and precision. (Useful for specific domains like finance).
- **Floating-Point:** Allow the binary point to "float" using scientific notation. This is the standard approach for general scientific computing.

Beyond Integers: Fractional Numbers

How do we store numbers that are not integers (e.g., 0.25, π)?

The Binary Point

We use a binary point like a decimal point.

- Columns right of the point: 2^{-1} (0.5), 2^{-2} (0.25), ...

Example: 2.25 (Decimal) = 10.01 (Binary).

Fixed-Point vs. Floating-Point

- **Fixed-Point:** Fix the location of the binary point. Limited range and precision. (Useful for specific domains like finance).
- **Floating-Point:** Allow the binary point to "float" using scientific notation. This is the standard approach for general scientific computing.

Scientific Notation in Binary

Decimal Scientific Notation

$$1234.5 = 1.2345 \times 10^3 \text{ (Mantissa} \times \text{Base}^{\text{Exponent}})$$

Binary Scientific Notation

$$10.01_2 = 1.001_2 \times 2^1$$

Normalisation

Numbers are **normalised** so that there is always exactly one '1' before the binary point.

The Standard: IEEE 754

Defines how to store the sign, mantissa (significand), and exponent within a fixed number of bits (e.g., 32 or 64).

Scientific Notation in Binary

Decimal Scientific Notation

$$1234.5 = 1.2345 \times 10^3 \text{ (Mantissa} \times \text{Base}^{\text{Exponent}})$$

Binary Scientific Notation

$$10.01_2 = 1.001_2 \times 2^1$$

Normalisation

Numbers are **normalised** so that there is always exactly one '1' before the binary point.

The Standard: IEEE 754

Defines how to store the sign, mantissa (significand), and exponent within a fixed number of bits (e.g., 32 or 64).

Scientific Notation in Binary

Decimal Scientific Notation

$$1234.5 = 1.2345 \times 10^3 \text{ (Mantissa} \times \text{Base}^{\text{Exponent}})$$

Binary Scientific Notation

$$10.01_2 = 1.001_2 \times 2^1$$

Normalisation

Numbers are **normalised** so that there is always exactly one '1' before the binary point.

The Standard: IEEE 754

Defines how to store the sign, mantissa (significand), and exponent within a fixed number of bits (e.g., 32 or 64).

Scientific Notation in Binary

Decimal Scientific Notation

$$1234.5 = 1.2345 \times 10^3 \text{ (Mantissa} \times \text{Base}^{\text{Exponent}})$$

Binary Scientific Notation

$$10.01_2 = 1.001_2 \times 2^1$$

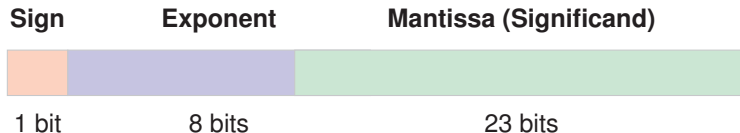
Normalisation

Numbers are **normalised** so that there is always exactly one '1' before the binary point.

The Standard: IEEE 754

Defines how to store the sign, mantissa (significand), and exponent within a fixed number of bits (e.g., 32 or 64).

IEEE 754 Structure (32-bit Single Precision)



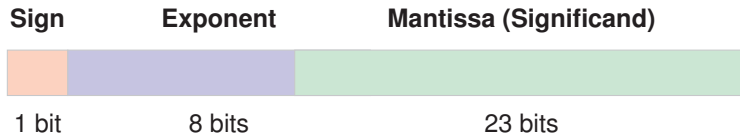
1. Sign Bit

0 for positive, 1 for negative.

2. Mantissa (The Hidden Bit)

Since normalised binary numbers (except zero) always start with $1.xxxx\dots$, we do not store the leading '1'. It is implied, effectively granting 24 bits of precision from 23 stored bits.

IEEE 754 Structure (32-bit Single Precision)



1. Sign Bit

0 for positive, 1 for negative.

2. Mantissa (The Hidden Bit)

Since normalised binary numbers (except zero) always start with $1.xxxx\dots$, we do not store the leading '1'. It is implied, effectively granting 24 bits of precision from 23 stored bits.

IEEE 754 Structure (32-bit Single Precision)

1. Sign Bit

0 for positive, 1 for negative.

2. Mantissa (The Hidden Bit)

Since normalised binary numbers (except zero) always start with 1.xxxx..., we do not store the leading '1'. It is implied, effectively granting 24 bits of precision from 23 stored bits.

3. Exponent (The Bias)

To represent negative exponents, IEEE 754 uses a **Bias** (127 for 32-bit).

$$\text{Stored Value} = \text{Actual Exponent} + 127$$

Why? This ensures the exponent field is always positive unsigned integer. This allows processors to compare floating-point numbers using simple integer comparison hardware.

IEEE 754 Example Conversion

Example: 2.25 (Decimal)

Binary: 10.01_2

Normalized: $1.001_2 \times 2^1$

Sign: Positive (0).

Exponent: 1. Stored value = $1 + 127 = 128$. Binary: 10000000.

Mantissa: .001. (The leading 1. is implied).



Resulting 32-bit Pattern

0 10000000 001000000000000000000000

IEEE 754 Example Conversion

Example: 2.25 (Decimal)

Binary: 10.01_2

Normalized: $1.001_2 \times 2^1$

Sign: Positive (0).

Exponent: 1. Stored value = $1 + 127 = 128$. Binary: 10000000.

Mantissa: .001. (The leading 1. is implied).



Resulting 32-bit Pattern

0 10000000 001000000000000000000000

IEEE 754 Special Values

The standard reserves specific exponent patterns for special values.

Zero (+0 and -0)

- Exponent = All 0s, Mantissa = All 0s.

Infinity ($\pm\infty$)

- Exponent = All 1s, Mantissa = All 0s.
- Result of operations like division by zero ($1/0$) or overflow.

NaN (Not a Number)

- Exponent = All 1s, Mantissa = Non-zero.
- Result of invalid operations (e.g., $\sqrt{-1}$, $0/0$, $\infty - \infty$).

IEEE 754 Special Values

The standard reserves specific exponent patterns for special values.

Zero (+0 and -0)

- Exponent = All 0s, Mantissa = All 0s.

Infinity ($\pm\infty$)

- Exponent = All 1s, Mantissa = All 0s.
- Result of operations like division by zero ($1/0$) or overflow.

NaN (Not a Number)

- Exponent = All 1s, Mantissa = Non-zero.
- Result of invalid operations (e.g., $\sqrt{-1}$, $0/0$, $\infty - \infty$).

IEEE 754 Special Values

Zero (+0 and -0)

- Exponent = All 0s, Mantissa = All 0s.

Infinity ($\pm\infty$)

- Exponent = All 1s, Mantissa = All 0s.
- Result of operations like division by zero ($1/0$) or overflow.

NaN (Not a Number)

- Exponent = All 1s, Mantissa = Non-zero.
- Result of invalid operations (e.g., $\sqrt{-1}$, $0/0$, $\infty - \infty$).

Floating Point in Python

Python's 'float' type

Python uses the **64-bit (Double Precision)** IEEE 754 format.

- 1 Sign bit, 11 Exponent bits, 52 Mantissa bits (53 effective precision).

Inspecting Properties

We can inspect the characteristics using `sys.float_info`.

```
1 print(sys.float_info)
```

Key Characteristics (64-bit)

- `mant_dig=53`: 53 bits of precision.
- `epsilon=2.22...e-16`: Machine Accuracy.

Floating Point in Python

Python's 'float' type

Python uses the **64-bit (Double Precision)** IEEE 754 format.

- 1 Sign bit, 11 Exponent bits, 52 Mantissa bits (53 effective precision).

Inspecting Properties

We can inspect the characteristics using `sys.float_info`.

```
print(sys.float_info)
```

Key Characteristics (64-bit)

- `mant_dig=53`: 53 bits of precision.
- `epsilon=2.22...e-16`: Machine Accuracy.

Floating Point in Python

Python's 'float' type

Python uses the **64-bit (Double Precision)** IEEE 754 format.

- 1 Sign bit, 11 Exponent bits, 52 Mantissa bits (53 effective precision).

Inspecting Properties

We can inspect the characteristics using `sys.float_info`.

```
3 print(sys.float_info)
```

Key Characteristics (64-bit)

- `mant_dig=53`: 53 bits of precision.
- `epsilon=2.22...e-16`: Machine Accuracy.

Numerical Errors and Machine Arithmetic

Numerical Errors and Machine Arithmetic

The Fundamental Limitation of Floating Point

The Core Issue

Floating-point numbers have finite precision and cannot perfectly represent all real numbers.

The Problem of Binary Representation

Many common decimal fractions cannot be represented exactly in binary with a finite number of bits.

- Example: 0.1 (Decimal)
- Binary: 0.00011001100110011... (It repeats infinitely, like 1/3 in decimal).

The Consequence: Rounding Errors

When the binary representation is truncated to fit the finite mantissa, the value is rounded. This introduces small errors that accumulate during calculations.

The Fundamental Limitation of Floating Point

The Core Issue

Floating-point numbers have finite precision and cannot perfectly represent all real numbers.

The Problem of Binary Representation

Many common decimal fractions cannot be represented exactly in binary with a finite number of bits.

- Example: 0.1 (Decimal)
- Binary: 0.00011001100110011... (It repeats infinitely, like 1/3 in decimal).

The Consequence: Rounding Errors

When the binary representation is truncated to fit the finite mantissa, the value is rounded. This introduces small errors that accumulate during calculations.

The Fundamental Limitation of Floating Point

The Core Issue

Floating-point numbers have finite precision and cannot perfectly represent all real numbers.

The Problem of Binary Representation

Many common decimal fractions cannot be represented exactly in binary with a finite number of bits.

- Example: 0.1 (Decimal)
- Binary: 0.00011001100110011... (It repeats infinitely, like 1/3 in decimal).

The Consequence: Rounding Errors

When the binary representation is truncated to fit the finite mantissa, the value is rounded. This introduces small errors that accumulate during calculations.

Absolute vs. Relative Error

To understand the impact, we distinguish between two types of error measurement. Let x be the true value and $\tilde{x}$ be the approximation.

Absolute Error

$$\Delta x = \tilde{x} - x$$

- Measures the raw difference.

Relative Error

$$\frac{\Delta x}{x} = \frac{\tilde{x} - x}{x} \text{ (assuming } x \neq 0 \text{)}$$

- Measures the error relative to the magnitude of the true value.
- In numerical analysis, the relative error is generally more informative.

Absolute vs. Relative Error

To understand the impact, we distinguish between two types of error measurement. Let x be the true value and $\tilde{x}$ be the approximation.

Absolute Error

$$\Delta x = \tilde{x} - x$$

- Measures the raw difference.

Relative Error

$$\frac{\Delta x}{x} = \frac{\tilde{x} - x}{x} \text{ (assuming } x \neq 0 \text{)}$$

- Measures the error relative to the magnitude of the true value.
- In numerical analysis, the relative error is generally more informative.

Machine Accuracy (Epsilon)

Machine Accuracy (or Machine Epsilon, ϵ) quantifies the limit of the floating-point system's precision due to rounding.

Definition

Machine Epsilon (ϵ) is the smallest positive number such that $1 + \epsilon > 1$ in the floating-point arithmetic.

Significance

It provides an upper bound on the relative error introduced by rounding a single number or a single operation.

$$\left| \frac{\text{rd}(x) - x}{x} \right| \leq \epsilon$$

For IEEE 754 Double Precision (Python), $\epsilon \approx 2.22 \times 10^{-16}$.

Machine Accuracy (Epsilon)

Machine Accuracy (or Machine Epsilon, ϵ) quantifies the limit of the floating-point system's precision due to rounding.

Definition

Machine Epsilon (ϵ) is the smallest positive number such that $1 + \epsilon > 1$ in the floating-point arithmetic.

Significance

It provides an upper bound on the relative error introduced by rounding a single number or a single operation.

$$\left| \frac{\text{rd}(x) - x}{x} \right| \leq \epsilon$$

For IEEE 754 Double Precision (Python), $\epsilon \approx 2.22 \times 10^{-16}$.

Computing Machine Accuracy (machine_accuracy.py)

We can empirically determine ϵ using a simple algorithm.

Listing 2: machine_accuracy.py

```
1 import sys
2
3 def compute_machine_accuracy():
4     epsilon = 1.0
5     # Keep halving epsilon until the system can no longer
6     # distinguish between 1.0 and 1.0 + epsilon
7     while 1.0 + epsilon > 1.0:
8         epsilon /= 2.0
9     # The actual epsilon is the last value that was distinguishable
10    return 2.0 * epsilon
11
12 # print(f"Computed accuracy: {compute_machine_accuracy()}")
13 # print(f"sys.float_info.epsilon: {sys.float_info.epsilon}")
```

The Impact of Rounding Errors (float_round.py)

Rounding errors can lead to unexpected results, especially during conversions.

The Experiment

Calculate $(i/100.0) * 100.0$ and convert back to an integer. We expect i .

Listing 3: float_round.py

```
1 i = 29
2
3 # 100 is not a power of 2, causing representation errors
4 result_float = 100 * (float(i) / 100)
5 # result_float is actually 28.999999999999996447...
6 # Convert back to integer
7 a = int(result_float)
8 # Output: 29 != 28
```

The Impact of Rounding Errors (float_round.py)

The Explanation

- 1 29/100 cannot be represented exactly in binary float, so it is rounded slightly down.
- 2 'int()' conversion in Python **truncates toward zero** (it does not round to the nearest integer). 'int(28.999...)' is 28.

The Importance of the Base (float_round2.py)

Powers of Two

If we use divisors that are powers of two (e.g., 128), the binary representation is exact (it only involves changing the exponent), and this specific rounding error is avoided.

Listing 4: float_round2.py

```
1 i = 29
2 # 128 is 2^7. Division by 128 is exact in binary float.
3 result_float = 128 * (float(i) / 128)
4 a = int(result_float)
5 # Output: 29 == 29
```


The Importance of the Base (float_round2.py)

Listing 5: float_round2.py

```
1 i = 29
2 # 128 is 2^7. Division by 128 is exact in binary float.
3 result_float = 128 * (float(i) / 128)
4 a = int(result_float)
5 # Output: 29 == 29
```

The Lesson on Comparisons

Never compare floats for exact equality ('=='). Always check if they are within a small tolerance, e.g. $|x - y| < \text{tolerance}$.

Error Propagation and Amplification Factors

How do errors in the input variables x_j affect the result of a function $y_i = f_i(x)$?

Error Propagation Formula (Approximation)

The relative error of the output is approximately bounded by the sum of the relative errors of the inputs, multiplied by amplification factors (k_{ij}):

$$\left| \frac{\Delta y_i}{y_i} \right| \lesssim \sum_{j=1}^m k_{ij} \left| \frac{\Delta x_j}{x_j} \right|$$

Error Propagation and Amplification Factors

Error Propagation Formula (Approximation)

The relative error of the output is approximately bounded by the sum of the relative errors of the inputs, multiplied by amplification factors (k_{ij}):

$$\left| \frac{\Delta y_i}{y_i} \right| \lesssim \sum_{j=1}^m k_{ij} \left| \frac{\Delta x_j}{x_j} \right|$$

Amplification Factors (Condition Numbers)

k_{ij} quantify how sensitive the output is to changes in the input.

$$k_{ij}(x) = \left| \frac{x_j}{f_i(x)} \frac{\partial f_i}{\partial x_j}(x) \right|$$

If k_{ij} is large, the problem is **ill-conditioned**.

Error Propagation and Amplification Factors

Amplification Factors (Condition Numbers)

k_{ij} quantify how sensitive the output is to changes in the input.

$$k_{ij}(x) = \left| \frac{x_j}{f_i(x)} \frac{\partial f_i}{\partial x_j}(x) \right|$$

If k_{ij} is large, the problem is **ill-conditioned**.

Intuition

Don't panic at the ∂ symbol!

- Think of $\frac{\partial f}{\partial x}$ simply as the **Sensitivity Factor**.
- It asks: "If I nudge the input x by a tiny amount, does the output $f(x)$ jump wildly?"
- **High Condition Number** = High Sensitivity = Dangerous for Accuracy.

Conditioning Examples

Well-Conditioned Operations

- Multiplication ($y = x_1 x_2$) and Division ($y = x_1 / x_2$).
- Amplification factors $k_{ij} = 1$. Relative errors are not significantly amplified.

Potentially Ill-Conditioned Operations

Addition and Subtraction ($y = x_1 + x_2$).

$$k_1 = \left| \frac{x_1}{x_1 + x_2} \right|$$

Conditioning Examples

Well-Conditioned Operations

- Multiplication ($y = x_1 x_2$) and Division ($y = x_1 / x_2$).
- Amplification factors $k_{ij} = 1$. Relative errors are not significantly amplified.

Potentially Ill-Conditioned Operations

Addition and Subtraction ($y = x_1 + x_2$).

$$k_1 = \left| \frac{x_1}{x_1 + x_2} \right|$$

Conditioning Examples

Potentially Ill-Conditioned Operations

Addition and Subtraction ($y = x_1 + x_2$).

$$k_1 = \left| \frac{x_1}{x_1 + x_2} \right|$$

The Danger Zone

The amplification factor becomes very large if the magnitude of the sum $|x_1 + x_2|$ is much smaller than the magnitudes of the operands $|x_1|$ and $|x_2|$. This occurs when $x_1 \approx -x_2$.

Catastrophic Cancellation

The severe amplification of error caused by the subtraction of two nearly equal numbers is known as **Catastrophic Cancellation**.

Example (Error Amplification)

- Calculation A: $1.14 - 1.03 = 0.11$
- Calculation B (1% perturbation): $1.15 - 1.02 = 0.13$

The input error is $\approx 1\%$. The output relative error is $\frac{0.13 - 0.11}{0.11} \approx 18\%$.

The Consequence

Cancellation leads to a significant loss of precision. Avoiding cancellation is a fundamental goal in numerical algorithm design.

Catastrophic Cancellation

The severe amplification of error caused by the subtraction of two nearly equal numbers is known as **Catastrophic Cancellation**.

Example (Error Amplification)

- Calculation A: $1.14 - 1.03 = 0.11$
- Calculation B (1% perturbation): $1.15 - 1.02 = 0.13$

The input error is $\approx 1\%$. The output relative error is $\frac{0.13 - 0.11}{0.11} \approx 18\%$.

The Consequence

Cancellation leads to a significant loss of precision. Avoiding cancellation is a fundamental goal in numerical algorithm design.

Catastrophic Cancellation

The severe amplification of error caused by the subtraction of two nearly equal numbers is known as **Catastrophic Cancellation**.

Example (Error Amplification)

- Calculation A: $1.14 - 1.03 = 0.11$
- Calculation B (1% perturbation): $1.15 - 1.02 = 0.13$

The input error is $\approx 1\%$. The output relative error is $\frac{0.13 - 0.11}{0.11} \approx 18\%$.

The Consequence

Cancellation leads to a significant loss of precision. Avoiding cancellation is a fundamental goal in numerical algorithm design.

Non-Associativity of Machine Arithmetic

A consequence of finite precision and rounding is that standard mathematical laws may not hold exactly.

Floating-Point Arithmetic is NOT Associative

$(a \oplus b) \oplus c$ is not necessarily equal to $a \oplus (b \oplus c)$. The order of operations matters.

Non-Associativity of Machine Arithmetic

Floating-Point Arithmetic is NOT Associative

$(a \oplus b) \oplus c$ is not necessarily equal to $a \oplus (b \oplus c)$. The order of operations matters.

Example (Decimal Arithmetic, 2-digit Mantissa)

$a = 0.75, b = 0.055, c = -0.80$. True result: 0.005.

Order 1: $(a \oplus b) \oplus c$

- $0.75 + 0.055 = 0.805$. Rounded (rd): 0.81
- $0.81 - 0.80 = 0.01$. (100% Relative Error)

Order 2: $a \oplus (b \oplus c)$

- $0.055 - 0.80 = -0.745$. Rounded (rd): -0.75
- $0.75 + (-0.75) = 0.00$. (100% Relative Error)

Simulating Floating-Point Behaviour (float_operations.py)

We can use tools to simulate the effects of different mantissa lengths. The provided 'DecimalCustomFloat' class simulates decimal floating-point arithmetic with adjustable precision, allowing us to visualise these errors.

Listing 6: Simulating the Non-Associativity Example (Conceptual)

```
# Assuming DecimalCustomFloat (DCF) is imported from
    float_operations.py
2 # Initialise with 2-digit mantissa (m=2)
3 a = DCF(0.75, mantissa_digits=2)
4 b = DCF(0.055, mantissa_digits=2)
5 c = DCF(-0.80, mantissa_digits=2)
6 # Option 1: (a+b) rounds 0.805 to 0.81 before subtraction
7 result1 = (a + b) + c # Output: 0.01
8 # Option 2: (b+c) rounds -0.745 to -0.75 before addition
9 result2 = a + (b + c) # Output: 0.00
```

Simulating Floating-Point Behaviour (float_operations.py)

Application

Simulating lower precision (e.g., using 'CustomFloat' for binary simulation) helps test the numerical stability of algorithms, especially relevant for hardware with lower precision (e.g., GPUs in ML).

Data Serialisation

Data Serialisation

Storing Complex Data Structures

How do we store complex Python objects like lists, dictionaries, or custom classes?

The Need for Serialization

We need a process to convert the in-memory data structure into a linear sequence of bytes (for storage) and reconstruct the object later. This is called **Serialization** (or Marshalling).

Formats

- **Text-based (e.g., JSON):** Human-readable, interoperable, but can be verbose.
- **Binary (e.g., Pickle):** Compact, efficient, but often language-specific.

Storing Complex Data Structures

How do we store complex Python objects like lists, dictionaries, or custom classes?

The Need for Serialization

We need a process to convert the in-memory data structure into a linear sequence of bytes (for storage) and reconstruct the object later. This is called **Serialization** (or Marshalling).

Formats

- **Text-based (e.g., JSON):** Human-readable, interoperable, but can be verbose.
- **Binary (e.g., Pickle):** Compact, efficient, but often language-specific.

Python Example: Pickle

The 'pickle' module provides a way to serialise Python objects into a binary format.

Serialisation (Dumping)

Listing 7: pickle_example.py (excerpt)

```
1 import pickle
2
3 list1 = [1, 2, 3]
4 list2 = ["Hello", "World"]
5 # Complex structure with nested references
6 list3 = [list1, list2, list1, list2]
7 # Open file in Write Binary ('wb') mode
8 with open('data.pickle', 'wb') as outfile:
9     pickle.dump(list3, outfile)
```

Python Example: Pickle

Deserialisation (Loading)

Listing 8: unpickle_example.py (excerpt)

```
1 import pickle
2
3 # Open file in Read Binary ('rb') mode
4 with open('data.pickle', 'rb') as infile:
5     loaded_list = pickle.load(infile)
6 print(loaded_list)
```

CRITICAL Security Warning

Never unpickle data from an untrusted source. A malicious pickle file can execute arbitrary code when loaded. For data interchange, prefer safer formats like JSON.

Python Example: Pickle

Deserialisation (Loading)

Listing 9: unpickle_example.py (excerpt)

```
1 import pickle
2
3 # Open file in Read Binary ('rb') mode
4 with open('data.pickle', 'rb') as infile:
5     loaded_list = pickle.load(infile)
6 print(loaded_list)
```

CRITICAL Security Warning

Never unpickle data from an untrusted source. A malicious pickle file can execute arbitrary code when loaded. For data interchange, prefer safer formats like JSON.

Summary and Key Takeaways I

Data Representation

- **Text Encoding:** Evolved from ASCII to Unicode (Code Points). UTF-8 is the standard variable-length encoding balancing efficiency and compatibility.
- **Integers:** Requires defining Size, Endianness (Big vs. Little), and Signedness (Two's Complement is the standard).
- **Floating Point (IEEE 754):** Uses binary scientific notation (Sign, Exponent, Mantissa). Allows a wide range but has finite precision.

Numerical Errors

Summary and Key Takeaways II

- **Inexactness:** Many decimal fractions cannot be represented exactly, leading to rounding errors (bounded by Machine Epsilon ϵ).
- **Error Analysis:** We use relative error and condition numbers (Amplification Factors) to analyse how errors propagate.
- **Catastrophic Cancellation:** Subtraction of nearly equal numbers is ill-conditioned and drastically amplifies relative error.
- **Non-Associativity:** The order of floating-point operations matters.